

LAB2

Task 1:

Create a class named as **Date** having following private attributes:

1. **day(int)**
2. **month(int)**
3. **year(int)**

Now write the following for the above mentioned class:

- Write a function **init** which should receive three integers as parameters (**_day**, **_month** and **_year**) and initialize all the attributes with the respective values received in the parameters.
Remember: Validate the values in the above function. For eg: month can never be negative etc.
- Write a non-returning **display** function to print the attributes of the class.

Now write a program to create two objects of Date in main(). In one you have to ask user to enter his birthDate and in other the currentDate. Display the birthdate and currentDate of the user by display function. And then calculate age of the user from the objects and print it.

Task 2:

Create a class named as **Employee** having following private attributes:

1. **employeeCode(int)**
2. **ageAtJoining(int)**
3. **currentAge(int)**

Now write the following for the above mentioned class:

- Write a function **init** which should receive three integers as parameters (**_employeeCode**, **_ageAtJoining** and **_currentAge**) and initialize all the attributes with the respective values received in the parameters.
Remember: Validate the values in the above function. For eg: ageAtJoining can never be negative etc.
- Write a non-returning **display** function to print the attributes of the class.
- Write a function **calculateTenure** which calculates the tenure of the employee and returns it.

Write a program to create five objects of employee with different data.

Display the details of those employees whose tenure is 2 or more than 2 years.

Task 3:

Create a class named as **Triangle** having following private attributes:

1. **base (double)**
2. **height(double)**

Now write the following for the above mentioned class:

- Write a function **init** which should receive two doubles as parameters (**_base** and **_height**) and initialize all the attributes with the respective values received in the parameters.
Remember: Validate the values in the above function. For eg: height can never be negative etc.
- Write a non-returning **display** function to print the attributes of the class.
- Write a function **calculateArea** which calculates the area of the triangle and returns it.

Write a program to create **a five objects** of **Triangle** with different data. Display the attributes of the triangle through display function having greatest area (use function written in the class to calculate area).

Task 4:

Create a class named as **ComplexNumber** having following private attributes:

1. **realPart(double)**
2. **imaginaryPart(double)**

Now write the following for the above mentioned class:

- Write a function **init** which should receive two doubles as parameters (**_realPart** and **_imaginaryPart**) and initialize all the attributes with the respective values received in the parameters.
Remember: Validate the values in the above function.
- Write a non-returning **display** function to print the attributes of the class in the format given below.

Write a program to create **a five objects** of **ComplexNumber** with different data. And then display all the complex number objects in the following format.

Output Format: 3+2i

LAB3

Task 1:

Define a class **Cat** having attributes

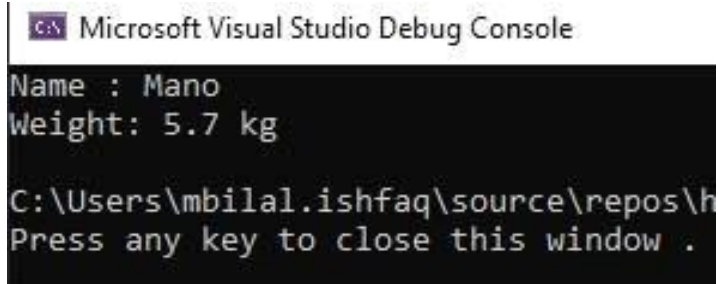
- **name (char *)**
- **weight (double)**

Now do the following operations on the above-mentioned class:

- Write **parameterized constructor with default arguments and copy the constructor** (with an output statement to check the lifetime scope of the object).
- Write a **destructor** (with an output statement to check the lifetime scope of the object).
- Write separate setter (mutator) functions for each attribute to set value (**with no memory leakage**).
- Write separate getter (accessor) functions for each attribute to get value (**which should never return the original memory handler**).
- Write a **display()** function to display attributes of class Cat on screen.

Now write the main function to declare different objects of class Cat. Then call the display function against every object to see the information stored.

For Example:



```
Microsoft Visual Studio Debug Console
Name : Mano
Weight: 5.7 kg
C:\Users\mbilal.ishfaq\source\repos\h
Press any key to close this window .
```

Task 2:

Define a class **Student** having attributes

- name (char *)
- registrationNumber (char *)
- CGPA (double)
- dateOfBirth (Date)

Hint: Use a separate class of Date to store it.

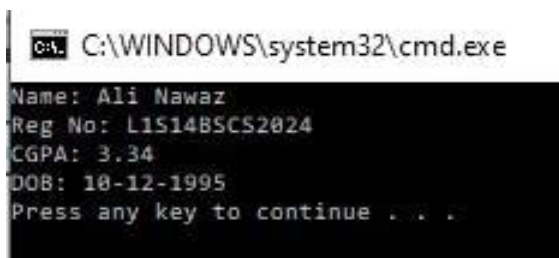
Now do the following operations on above-mentioned class:

- ☐ Write **parameterized constructor and copy the constructor** (with an output statement to check the lifetime scope of the object).
- ☐ Write a **destructor** (with an output statement to check the life time scope of object).
- ☐ Write separate setter (mutator) functions for each attribute to set value (**with no memory leakage**).
- ☐ Write separate getter (accessor) functions for each attribute to get value (**which should never return the original memory handler**).
- ☐ Write a **display()** function to display attributes of class Employee on screen.

Hint: For displaying the date call the display function of class Date.

Now write the main function to declare different objects of class Student. Then call the display function against every object to see the information stored.

For Example:



```
C:\WINDOWS\system32\cmd.exe
Name: Ali Nawaz
Reg No: L1S14BSC52024
CGPA: 3.34
DOB: 10-12-1995
Press any key to continue ...
```

Task 3:

Define a class **Employee** having attributes

- name (char *)
- employeeCode (char *)
- dateOfJoining (Date)

Hint: Use a separate class of Date to store it.

Now do the following operations on above mentioned class:

- Write **parameterized constructor with default arguments and copy constructor** (with an output statements to check the life time scope of object).
- Write a **destructor** (with an output statement to check the life time scope of object).
- Write separate setter (mutator) functions for each attribute to set value (**with no memory leakage**).
- Write separate getter (accessor) functions for each attribute to get value (**which should never return the original memory handler**).
- Write a **display()** function to display attributes of class Employee on screen.

Hint: For displaying date call display function of class Date.

Now write a main function to create three different objects of Employee with different data.

Then ask the user to enter current date. Then call display function against every object to see information stored and also print the total number of days of the employee in the organization.

For Example:

```
Microsoft Visual Studio Debug Console
Employee Code : 1416
Name: Bilal Ishfaq
Date of Joining : 5 - 3 - 2018
Total Number of Days : 605

C:\Users\mbilal.ishfaq\source\repos\hellow
Press any key to close this window . . .
```

LAB4 GRADED1

Lab Task 1:

Write the definition of a class, swimming Pool, to implement the properties of a swimming pool. Your class should have the instance variables to store the length (in feet), width (in feet), depth (in feet), the rate (in gallons per minute) at which the water is filling the pool, and the rate (in gallons per minute) at which the water is draining from the pool.

Now add setter and getter functions. Also add member functions to determine the amount of water needed to fill an empty or partially filled pool; determine the time needed to completely or partially fill or empty the pool; add or drain water for a specific amount of time.

Lab Task 2:

Design and implement a class dayType that implements the day of the week in a program. The class dayType should store the day, such as Sun for Sunday. Write setter and getter functions of the class dayType. Also add member functions to perform the following operations on an object of type dayType:

- a) Print the day.
- c) Return the next day.
- e) Return the previous day.
- f) Calculate and return the day by adding certain days to the current day. For example, if the current day is Monday and we add 4 days, the day to be returned is Friday. Similarly, if today is Tuesday and we add 13 days, the day to be returned is Monday.

Lab Task 3:

Write a C++ program to represent a bank account using class and objects. Display the details such as name, account number, account type and balance in the given account number. Add setter and getter functions. User can deposit and withdrawn money from the account.

Lab Task 4:

Create a class for traffic signal. use an object of the class to turn the green , red and yellow signals on and off.

Lab Task 5:

An automobile company has serial number for engine parts starting from AA0 to FF9. The other characteristics of parts to be specified as attributes are: Year of manufacture, material and quantity manufactured.

Write getters and setters for each characteristic. Make three objects and set the values of these characteristics through setters and display them through getters.

Lab Task 6:

A class named "employee" holds information like employee code, name, date of joining. Write a program to create three objects of employee and enter some data into it through setters. Make getters and setters for all employee information. Then ask the user to enter current date. Display the names of those employees whose tenure is 2 or more than 2 years according to the given current date only using getters.

Lab Task 7:

Write a program that compares two given dates. To store date use structure "date" that contains three members namely date, month and year. If the dates are equal then display message as "Equal" otherwise "Unequal". (You have to re-do the problem that you did in the previous lab. However, this time, all data members must be private and use appropriate getters, setters and functions)

Lab Task 8:

A class named "Card" holds information of card suit (i.e diamond, hearts, Spades, Clubs), card number (2-10), and face (front/back). Diamonds and hearts have red color; spaded and clubs have black color.

~~Write a program to create 5 objects of Card and set data by taking input from user (note that cin statement is never ever used in the setter function). Display information of the same cards. (Same number, same color, face=front)~~

LAB5 GRADED2

Task 1

Create class SavingsAccount. Use a variable annualInterestRate to store the annual interest rate for all account holders. Each object of the class contains a private instance variable savingsBalance indicating the amount the saver currently has on deposit. Provide method calculateMonthlyInterest to calculate the monthly interest by multiplying the savingsBalance by annualInterestRate divided by 12 this interest should be added to savingsBalance. Provide a method modifyInterestRate that sets the annualInterestRate to a new value. Write a program to test class SavingsAccount. Instantiate two savingsAccount objects, saver1 and saver2, with balances of \$2000.00 and \$3000.00, respectively. Set annualInterestRate to 4%, then calculate the monthly interest and print the new balances for both savers. Then set the annualInterestRate to 5%, calculate the next month's interest and print the new balances for both savers.

```
int main()
{
    SavingsAccount saver1;
    SavingsAccount saver2;  saver1.setSavingsBalance(2000.00);
    saver2.setSavingsBalance(3000.00);

    SavingsAccount::modifyInterestRate(0.04);
    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();

    cout << "New Balance for Saver1=\n" <<
    saver1.getSavingsBalance();
    cout << "New Balance for Saver2=\n" <<
    saver2.getSavingsBalance();

    SavingsAccount::modifyInterestRate(0.05);
    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();

    cout << "New Balance for Saver1=\n" <<
    saver1.getSavingsBalance();
    cout << "New Balance for Saver2=\n" <<
    saver2.getSavingsBalance();
    return 0;
}
```

Task 2

Create Time class making use of Copy constructor:

In this lab you will write a class called Time. The Time class will have three data members: hours, minutes, and seconds (all ints). Times will be stored using a 24 hour clock, so 10 AM is stored as 10 and 10 PM is stored as 22.

Your Time class should have the following member functions:

default constructor

This function initializes the Time object to contain 12 midnight (hour zero, minute zero, seconds zero).

constructor with three parms

This function has parms for the hour, minutes, and seconds. It initializes the Time object to the values given by the parms.

copy constructor

This function initializes the Time object to contain the same values as the parm.

setTime

This function has parms for the hour, minute, and second, and returns void. setTime sets the time to the values given by the parms.

showTime

This function has no parms and returns void. It prints the time using the normal 12 hour clock and AM/PM designation. For example, if hours is 21, minutes is 32 and seconds is 5, the time should be displayed as 9:32:05 PM. If hours is 5, minutes is 12, and seconds is 45, the time should be displayed as 5:12:45 AM. Don't forget to add a leading zero if the minutes and/or the seconds contains a value less than 10!

tick

This function has no parms and returns void. It increments the time by onesecond, adjusting the minutes and hours if necessary.

Write client code to test your Time class. Your main function should:

1. Create two Time objects and prompt the user to enter values for the Time objects.
2. Print the Time objects.
3. Print a message that tells which Time object contains a shorter distance.
4. Tick the first Time object 10 times, then print the first Time object.
5. Create a third Time object that is initialized by the default constructor.
6. Create a fourth Time object that is initialized to 10:56:05AM.
7. Create a fifth Time object that is initialized to be a copy of the fourth Time object.

Print the third, fourth, and fifth Time objects.

Task 3

Create class for items in a general store. The items will have the following characteristics.

- Item_code (int)
- Item_name (char *)
- Price; (int) • Expiration_date:
- Current_date:

Calculate the amount of days remaining till the expiration date. If the days remaining are less than 20 then the price of item should be reduced by 40%.

Along with parameterized constructor, the setter and getter functions you have to create a copy constructor(Deep copy).

In main function, create an object1 and initialize it with parameterized constructor. Create object2 and initlize it with object1. Change item name of object2 and print data of both objects.

Task 4

Next Bridge software house Lahore requires an application that calculates employee's increment based on the given criteria. For the purpose software house wants his software engineer to design the application using object oriented approach. The class"NextBridge" is required to create with attributes and functions as mentioned below.

- name:char*
- address:char*
- phoneNumber: char*
- level: char* //fresh,Junior,senior
- experience:int //in years
- currentSalary: double
- increment: double
// Constructors
// Destructor

// Get Functions

// Set Function

// Calculate increment

// Display Function (parameterized function)

Increment criteria	
Experience	Increment
0	0%
01- 02	10 %
02- 06	20%
06 to onward	40%

- When an object of the class is instantiated, it is always done by using an overloaded constructor having 7 parameters. Examples of such instantiations are given below: NextBridge obj("Name", "Address", "Ph:0300-123", "fresh", 2, 10000, 0);

- Create a copy constructor. Copy data of Object1 to object 2 using copy constructor. Write a test program that demonstrates capabilities of the class.
- Use demo main function to illustrate everything

Task 5:

Design and implement your own string library(string.h). This library should include at least 7 most commonly used functions of built in string library.

LAB6

Task 1:

Define a class **Car** having attributes

- name (char *)
- model (char*)
- noOfDoor (int*)

Now do the following operations on the above-mentioned class:

1. Write **default constructor, parameterized constructor, and copy constructor**.
2. Write separate setter functions for each attribute to set value (**with no memory leakage**).
3. Write separate getter functions for each attribute to get value.
4. Write a **destructor** (with an output statement to check the lifetime scope of the object).
5. Write a **display()** function to display attributes of class Car on screen.

Now write the main function to declare different objects of class Cat. Then call the display function against every object to see the information stored.

Task 2:

Define a class **Employee** having attributes

- name (char *)
- hourly_pay_rate (float*)
- working_hours(int*)

Now do the following operations on above-mentioned class:

1. Write **default constructor, parameterized constructor and copy constructor**.
2. Write separate setter functions for each attribute to set value (**with no memory leakage**).
3. Write separate getter functions for each attribute to get value.
4. Write a **destructor** (with an output statement to check the lifetime scope of object).
5. Write a **display()** function to display attributes of class Employee on screen.
6. Write an AddSal() function, which adds \$10 to the salary of the employee if it is less than \$500
7. Write an “AddWork()” function, which adds \$5 to the salary of the employee if the number of hours of work per day is more than 6 hours.

Task 3:

Write a C++ program to create a complex number class. Write getter setter, constructor, destructor, and all overloaded operators needed to run the given driver code.

```
class complex {
private:
    int real; int imag; public:

    //All functions that are needed.
};

int main()
{
    complex c1(5,7);           // for complex numbers, such as 5+7i
    complex c2(5);             // for numbers without imaginary part as 5 + 0i
    complex c3(); complex c4(); c3 =
    c1+c2; c4 = c1-c2; c5 = c3*c4; c6 =
    c4/c5; return 0;
}
```

LAB7

Task 1:

Let's create a class `Time` to work with a 12-hour formatted time having the following attributes:

```
int* hours int* minutes
int* seconds
```

Provide the following functions:

1. According to the rule of three:
 - i. Copy Constructor
 - ii. `operator =`
 - iii. Destructor
2. Overloaded constructor with default values
3. Getter of each attribute (setters are NOT required)
4. `operator ==`
5. `operator >`
6. As discussed in class, only using `operator ==` and `operator >`, provide implementation of:
 - i. `operator !=`
 - ii. `operator <`
 - iii. `operator <=`
 - iv. `operator >=`
7. `operator <<` //Example Output **2:13:47**
8. `operator >>`
9. `operator +` //to add two values of time
10. `operator -` //to subtract two values of time
11. `operator ++`
 - i. post-increment
 - ii. pre-increment
12. `operator --`
 - i. post-increment
 - ii. pre-increment

Make sure to check that the time remains valid in all the functions that are:

- A. `1 <= hours <=12`
- B. `0 <= minutes < 60`
- C. `0 <= seconds < 60`

Task 2:

Repeat task 1, to create a class for `Date`; however, no need to take pointer attributes; the attributes will therefore be:

```
int day; int month;
int year;
```

1. Rule of three is NOT required (we don't have any pointer attribute). Rule of three is required only if there is at least one pointer attribute.
2. Overloaded constructor with default values
3. Getter of each attribute (setters are NOT required)
4. `operator ==`
5. `operator >`
6. As discussed in class, only using `operator ==` and `operator >`, provide implementation of:
 - i. `operator !=`
 - ii. `operator <`
 - iii. `operator <=`
 - iv. `operator >=`
7. `operator <<` //Example Output: 30-4-2013
8. `operator >>`
9. `operator +` //to add integer days in a `Date`
Hint: The prototype should be:

```
Date operator + (int d) //where d is days
```

Example: **20-3-1985 + 16 days** will return date as: **5-4-1985**
10. `operator -` //to subtract two values of time
11. Hint: The prototype should be:

```
Date operator - (int d) //where d is days
```

Example: **20-3-1985 - 155 days** will return date as: **16-10-1984**
12. `operator ++`
 - i. post-increment
 - ii. pre-increment
13. `operator --`
 - i. post-increment
 - ii. pre-increment

Make sure to check that the date remains valid in all the functions that are:

- A. `1 <= day <= 31`
- B. `0 <= month <= 12`
- C. `0 <= year <= n` where `n` is a positive integer

LAB8

TASK 1

In this lab task you are going to create three classes

1. driver
2. rider
3. ride

Create a class **driver** with the following data members and methods:

Data members:

- driver's name (string)
- driver's contact Number (string)
- driver's address (string)
- driver's CNIC (string)
- trips completed (int)
- driver's rating (float)

Methods:

- default parametrized constructor (by default: store 0 in all numerical and NULL in all string variables)
- setter and constant getter functions for each data member.
- constant members function Driver's info. This function should display all the info related to driver.

Create a class **rider** with the following data members and methods:

Data members:

- rider's name (string)
- rider's contact (string)
- rider's rating (float)
- trips cancelled (int)

Methods:

- default paramterized constructor (by default: store 0 in all numerical and NULL in all string variables)
- setter and constant getter functions for each data member.
- constant member function Rider's info. This function should display all the info related to rider.

Create a class **ride** with the following data members and methods:

Data members:

- an object of driver
- an object of rider
- pickup location (string)
- destination (string)
- vehicle number (string)
- tracking id (int)
- category (string, provided below)

- base fare (float)
- per km charges (float, provided below)
- per min charges (float, provided below)
- promo code (string)
- discount (float)
- start time (preferably string that stores the time in 24hr clock e.g 1620 for 4:20PM)
- arrival time (preferably string that stores the time in 24hr clock e.g 1658 for 4:58PM)

Methods:

- default parameterized constructor (by default: store 0 in all numerical and NULL in all string variables)
- setter functions for each data member including data members of driver and rider class.
- constant getter function calculateBill.
 - if the promo code provided by the user matches with the string "pk8976" than provide 25% discount.
 - calculate the total time of journey by subtracting ending time from start time. since their data types are string so use function stoi to convert these string values into integers. if stoi function is not working in your compiler than execute your code on onlinegdb compiler.
- constant member function showTrip info which display all the trip related info e.g.,
 - total fare:
 - fare of distance: ○ fare of time: ○ discount
 - This function should also call the member functions of driver and rider info to display their data.

Description of fares on different categories:

Bike:

- base fare: 60
- per minute charges: 3.67
- per km charges: 6.17

Auto:

- Base Fare: Rs. 50.71
- Per KM: Rs.12.09
- Per Minute: Rs. 2.79

Mini:

- Base Fare: Rs. 65
- Per KM: Rs. 8.13
- Per Minute: Rs. 3.63

Go:

- Base Fare: Rs. 80
- Per KM: Rs. 10.16
- Per Minute: Rs. 4.54

UberX:

- Base Fare: Rs. 100
- Per KM: Rs. 15.16
- Per Minute: Rs. 8.54

LAB9

Task 1

Create a Class **ReleaseDate** having Date of Release of Product. The Date should be in format day-month-year.

Create another class **Product** having the name of Product (char *); price(int); and Date of Release.

Identify the relationship and rest of functions needed for both of the classes and implement them.

In main create object of Product to show the full detail of every object e.g

Shampoo 200 12-02-2019

Soap 50 13-02-2018

Lotion 450 01-12-2019

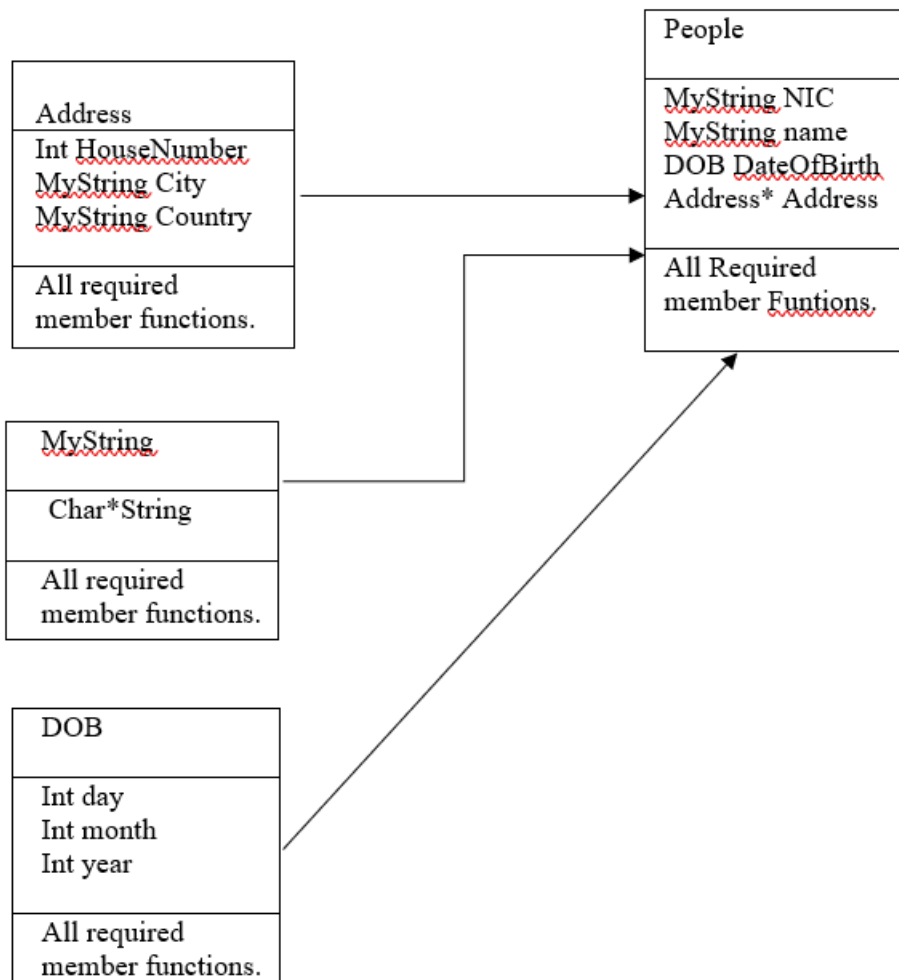
Task 2

Create a class People having name, NIC,Address and DOB as its attributes.

DOB object belongs to a class DOB ,having day-dat-year as its attributes.

Mystring class has char*String as its attribute and Address has HouseNumber, City, And Country.

Identify the relationship among all and implement all the required functions in all classes to make them functional.



In main create object of People to show the full detail of People object

Task 3:

Define a class **MyChar** having one private attributes: A single character variable.

Now add the following operations on above mentioned class MyChar:

1. Write parameterized constructor with default arguments.
2. Write separate setters for each attribute to set value.
3. Write separate getters for each attribute to get value.
4. Overload `cin>>` operator for MyChar.
5. Overload `cout<<` operator for MyChar.
6. A function that can convert the character stored in MyChar into lowerCase.
7. A function that can convert the character stored in MyChar into upperCase.

Then, define a class MyString having following attributes:

1. A **MyChar*** to make an array of MyChar.
2. A count to maintain the number of MyChar in the array.

Finally, add the following operations on above mentioned class MyString:

1. Write **parameterized constructor with default argument**.
2. Write a copy Constructor.
3. Write an assignment operator.
4. Write a destructor.
5. Overload `+` Operator to concatenate two MyString objects.
6. Overload **cin>>** operator for MyString.
7. Overload **cout<<** operator for MyString.
8. Overload pre-increment and post-increment operator which converts the MyString into upperCase.
9. Overload pre-decrement and post-decrement operator which converts the MyString into lowerCase.
10. Write a `print(const MyString&)` function which should be a **non-member function** and you are **not** allowed to return **the memory handler** anywhere in your programme.

Hint: Overload `[]` operator.

Remember:

There should be no memory leakage in your programme.

Use const qualifier wherever possible.

Your programme should compile and run properly.

Write a driver programme(`main()`) to test “all” functionalities and operators of MyString class on your own.

Task 1

Pakistan Post Office (PPO) offer a number of different shipping options, each package has specific costs. Create an inheritance hierarchy to represent various types of packages.

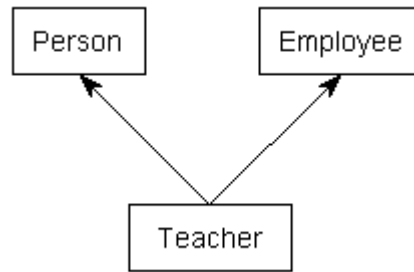
1. Use class **Package_PPO** as the base class of the hierarchy, then include classes **NormalPackage**, **TwoDayPackage** and **OvernightPackage** that derive from **Package_PPO**
2. Base class **Package_PPO** should include data members representing the name, address, city, state and ZIP code for both the sender and the recipient of the package, in addition to data members that store the weight (in Kg) and cost per KG to ship the package. **Package_PPO**'s constructor should initialize these data members. Ensure that the weight and cost per KG contain positive values.
3. **Package_PPO** should provide a public member function `calculateCost` that returns a double indicating the cost associated with shipping the package.
4. **NormalPackage** have all same attribute as base class but its `calculateCost` function should determine the cost by multiplying the weight by the cost per KG.
5. Derived class **TwoDayPackage** should inherit the functionality of base class **Package_PPO**, but also include a data member that represents a flat fee that the shipping company charges for two-day-delivery service. **TwoDayPackage**'s constructor should receive a value to initialize this data member. **TwoDayPackage** should redefine member function `calculateCost` so that it computes the shipping cost by adding the flat fee to the weight-based cost.
6. Class **OvernightPackage** should inherit directly from class **Package_PPO** and contain an additional data member representing an additional fee per KG charged for overnight-delivery service. **OvernightPackage** should redefine member function `calculateCost` so that it adds the additional fee per KG to the standard cost per KG before calculating the shipping cost.

Instructions:

- i. Create different type of Constructors according to question
- ii. Create Setter getter where necessary
- iii. Your design should use all concepts of OOP we have studied so far.

Task 2: MULTIPLE INHERITANCE is a part of Midterm

Provide the C++ implementation of the following.



The Person class has name and age as its attributes. It has an overloaded constructor to initialize the values and appropriate accessor and mutator methods. The Employee class has name of the employer and wage as its attributes with an overloaded constructor and appropriate accessor and mutator methods. The Teacher class is inherited from both Person and Employee class. This is known as MULTIPLE inheritance. You will do it like this in Teacher class:

```
class Teacher: public Person, public Employee
```

Note that teacher is inheriting from both Person and Employee. In multiple inheritance, there are more than 1 parents. In this example, there are two parents. There could have been more than 2 parent classes too.

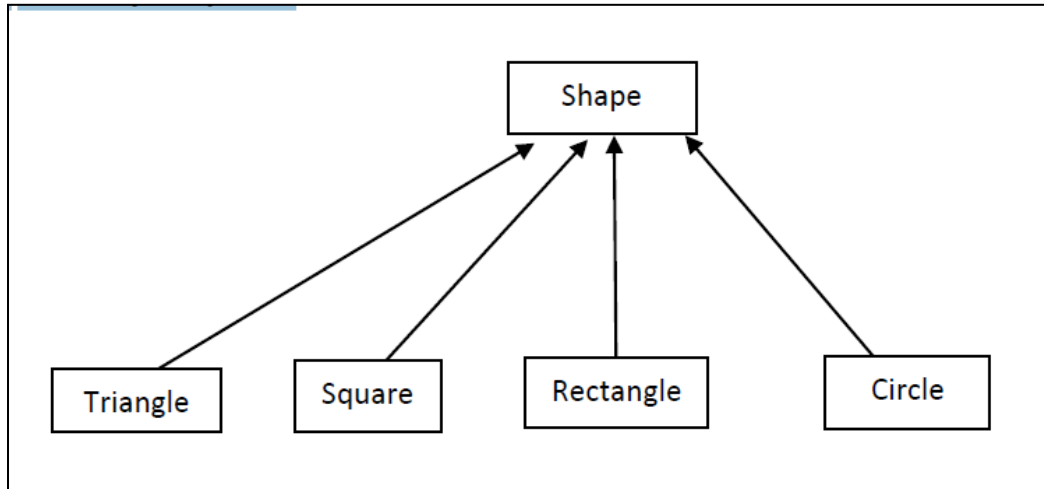
Teacher has an attribute of Pay Scale of type integer. It has overloaded constructor, appropriate accessor (getter), mutator (setter) methods and a display function to print the Name, Age, Name of Employer, Wage and Pay Scale of Teacher.

LAB11 GRADED4

Task:1 (done in file ----Accessible, Not Accessible)

Task 2:

Implement the following class hierarchy. Write a function **area()** to calculate the area of each object of any **Shape**.



For Triangle: $A = \frac{1}{2} * \text{length} * \text{width}$

For Square: $A = \text{length} * \text{length}$

For Rectangle: $A = \text{length} * \text{width}$

For Circle: $A = \pi r^2$

- All the attributes MUST be pointers.
- Make a parameterized constructor with default arguments that take parameters required in each class and use the base initializer list to call the parameterized constructor with default arguments of Base class
- Make a copy constructor and use the base initializer list to call the copy constructor of Base class. (Remember you can call constructors of the base class through initializer lists; this rule applies to copy constructors as well).
- Further, write a display function as well.

Do submit the screenshot of the output with your code.

Task 3:

Create a base class **Card** with the following attributes

- Card number : private
- Owner name : protected
- Expiry date : public

Derive the following classes from **Card**, with mentioned additional attributes

• **Calling card** (public inheritance)

- Amount : private
- Company name : private
- PIN : private

• **ID card** (protected Inheritance)

- CNIC Number : private
- Age : private

• **Driving license card** (private Inheritance)

- Driving license type (heavy, light, bike) : private
- Issued in city : private

Your tasks:

1. In the derived classes, write the getters and setters of every member variable (including the derived variables). You are not allowed to make any member function in the base class. A main() is required to add up some a card of each type, and then to display their information. The object of the base class will not be instantiated.
2. According to the rules of inheritance, clearly specify (by adding comments in the derived classes) which of the members are inherited and clearly mention their access specifiers in the derived classes.
3. Draw the UML diagram of the above-mentioned scenario in any software of your choice.

Task 4:

Create a base class, called BankAccount, and two additional classes (each derived from BankAccount), called SavingsAccount and CheckingAccount.

BankAccount:

- Title
- AccountNumber
- Balance
- Deposit()
- Withdraw()

SavingAccount:

- InterestRate
- CalculateInterest()

CheckingAccount:

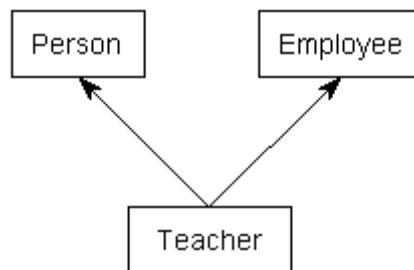
- fee charged per transaction

Class CheckingAccount should redefine member functions withdraw and deposit so that they subtract the fee from the account balance whenever either transaction is performed successfully.

You will then test the operations of each class in function main() to simulate the transactions of both a checking account and a savings account.

Task 5:

Provide the C++ implementation of the following.



The Person class has name and age as its attributes. It has an overloaded constructor to initialize the values and appropriate accessor and mutator methods. The Employee class has the name of the employer and wage as its attributes with an overloaded constructor and appropriate accessor and mutator methods.

The Teacher class is inherited from the Person and Employee class with an attribute of Pay Scale of type integer. It has an overloaded constructor, appropriate accessor, mutator methods, and a display function to print the Name, Age, Name of Employer, Wage, and Pay Scale of the Teacher.

LAB12 & LAB14

Problem 1:

Define an abstract base class shape that includes protected data members for area and volume of a shape, public methods for computing area and volume of a shape (make the functions virtual), and a display function to display the information about an object. Make this class abstract.

Derive a concrete class point from the shape class. This point class contains two protected data members that hold the position of point. Provide no-argument and 2-argument constructors. Override the appropriate functions of base class.

Derive a class Circle publicly from the point class. This class has a protected data member of radius. Provide a no-argument constructor to initialize the fields to some fixed values. Provide a 3-argument constructor to initialize the data members of Circle class to the values sent from outside. Override the methods of base class as required.

Derive another class Cylinder from the Circle class. Provide a protected data member for height of cylinder. Provide a no-argument constructor for initializing the data members to default values. Provide a 4-argument constructor to initialize x- and y-coordinates, radius, and height of cylinder. Override the methods of base class.

Write a driver program to check the polymorphic behavior of this class.

Problem 2:

Create a class named Pizza that stores information about a single pizza. It should contain the following:

- Private instance variables to store the size of the pizza (either small, medium, or large), the number of cheese toppings, the number of pepperoni toppings, and the number of ham toppings.
- Constructor(s) that set all of the instance variables.
- Public methods to get and set the instance variables.
- A public method named `calcCost()` that returns a double that is the cost of the pizza.
- Pizza cost is determined by:
 - Small: $\$10 + \2 per topping
 - Medium: $\$15 + \2 per topping
 - Large: $\$20 + \2 per topping
- public method named `getDescription()` that returns a String containing the pizza size, quantity of each topping.

Write test code to create several pizzas and output their descriptions. For example, a large pizza with one cheese, one pepperoni and two ham toppings should cost a total of \$22. Now Create a `PizzaOrder` class that allows up to three pizzas to be saved in an order. Each pizza saved should be a `Pizza` object. Create a method `calcTotal()` that returns the cost of order.

In the runner order two pizzas and return the total cost.

Problem 3:

Function Templates for Sorting Arrays: The function takes an array of elements of any data type and sorts them in ascending and descending order

Task 1

In geometry, a shape can be defined as the form of an object or its outline, outer boundary or outer surface. Everything we see in the world around us has a shape. We can find different basic shapes such as the two-dimensional square, rectangle, and triangle. The 2-Dimensional shape is flat. It has two dimensions, that is, length and width. Every Shape has some area which can be calculated using the mathematical area function of that particular shape. The color, overall size, and orientation called the non-defining attributes of a shape do not define or affect the shape in any way. These attributes can change without any effect on the shape.

Using the above scenario answer the following questions

1. Identify classes attributes and their respective methods.
2. Draw the class diagram.
3. Implement the classes.
4. Make the objects of each class in main function and call all relevant function.
5. Now make the polymorphic calls for all the methods by making the appropriate objects.

Task 2 - Function Templating

For this part of the lab make a template out of the `myMax` function and test it on different data types.

- Start with the repl code provided to you.
- Compile and run the program to see how it works.
- Make a template out of `myMax`. Don't forget the return type.
- Modify the prototype appropriately.
- Test your `myMax` template on int, double, and string types.

When you are done your output should resemble this:

```
The max of 3 and 5 is 5
The max of 5.6 and 7.3 is 7.3
```

Task 3:

Perform the following tasks.

1. An abstract class called Creature
2. Classes Player and Monster (derived from Creature)
3. Classes WildPig and Dragon (derived from Monster)

In the Creature class

– Define a `char*` member, `CreatureName`, to store the class Creature's name.

– Two functions

void DoAction() : Print the action of the object, and the actions have to be different from different classes.

void DrawOnScreen() : Print the object's name and call DoAction() belonging to the same class.

The class definition of Creature is:

```
class Creature {
public:
    Creature(string);
    virtual void DoAction()const=0;
    virtual void DrawOnScreen()const=0;
protected:
    string CreatureName;
};
```

Implement the class Player, Monster, Dragon and Wildpig so that when execution the following code, the counsel shows the execution result as the following:

Output:

Main Function:

```
int main(){
    Player hero("Kick_Ass");
    Monster mon("UFO");
    WildPig pig("I'm_Hungry");
    Dragon drag("I'm_the_Boss");
    Creature* object[4];
    object[0]=&hero;
    object[1]=&mon;
    object[2]=&pig;
    object[3]=&drag;
    object[0]->DrawOnScreen();
    object[1]->DrawOnScreen();
    object[2]->DrawOnScreen();
    object[3]->DrawOnScreen();
    return 0;}
```

```
Player <Kick_Ass> is attacking!!
Monster <UFO> is doing monster stuff!!
WildPig <I'm_Hungry> is Running!!
Dragon <I'm_the_Boss> is breathing Fire!!!
Press any key to continue . . . _
```